

Par NIDHAL JELASSI
jelassi.nidhal@gmail.com

ANGULAR

Chapitre 1 : Introduction et Environnement



PRÉ-REQUIS

- ▶ HTML 5 / CSS 3 / Bootstrap



- ▶ JavaScript



- ▶ Programmation Orientée Objet

POO

CE QUI NOUS ATTEND...

Partie 1 :

- ▶ Présentation de Angular
- ▶ Principe du SPA
- ▶ Installation de l'environnement
- ▶ Composants.
- ▶ Interaction entre les composants
- ▶ Directives et pipes

Partie 2 :

- ▶ Les services et l'injection de dépendances
- ▶ DI hiérarchique
- ▶ Signals
- ▶ Routing et lazy loading
- ▶ Navigation
- ▶ Formulaires pilotés par le template.

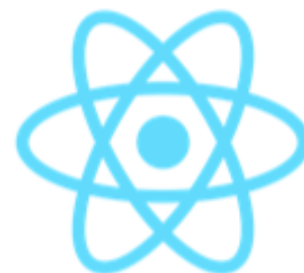
CE QUI NOUS ATTEND...

Partie 3 :

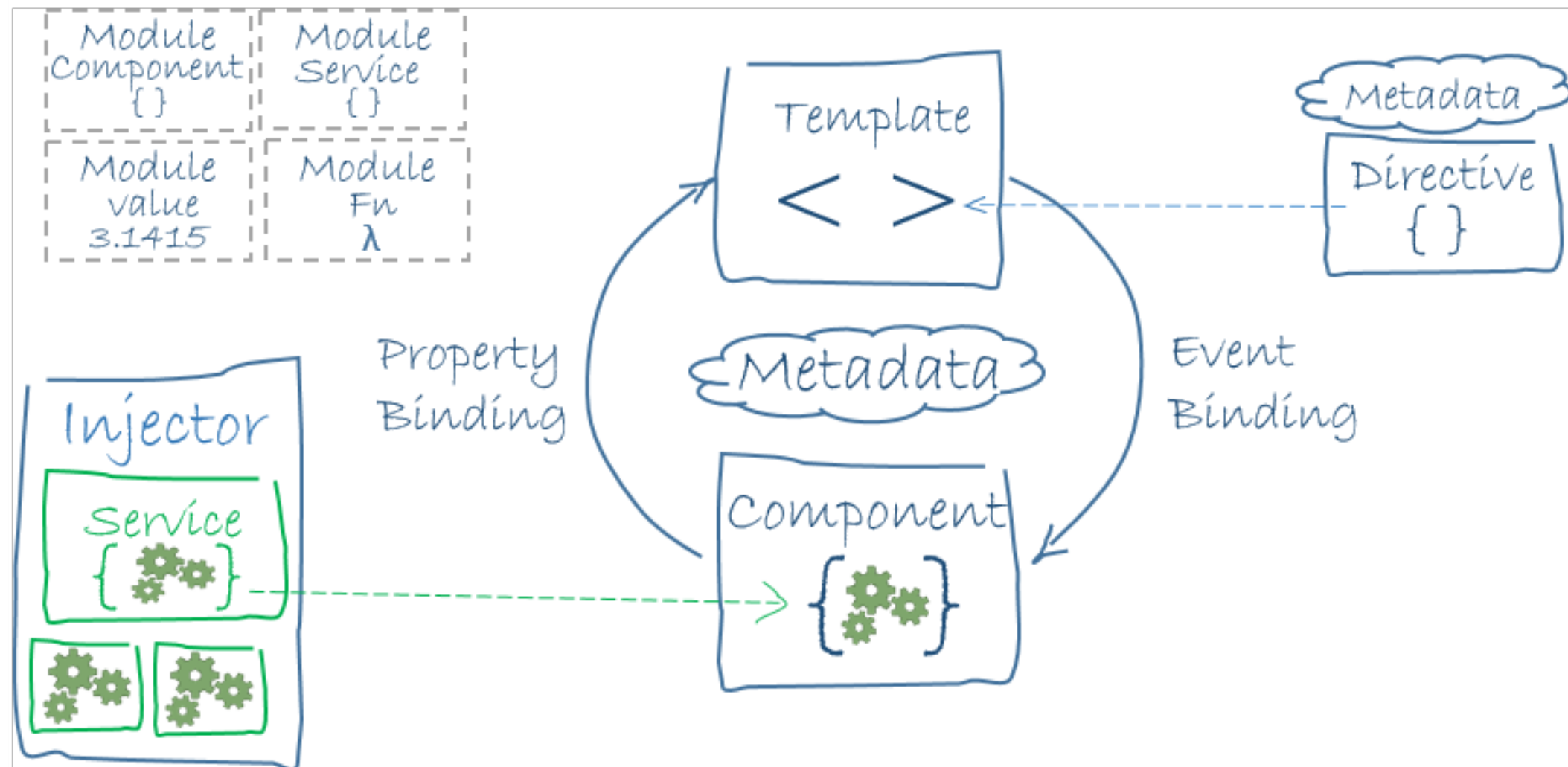
- ▶ Reactive Forms
- ▶ Les observables (RxJS)
- ▶ Mise en place du Backend
- ▶ Effectuer des requêtes HTTP standards.
- ▶ Intercepteurs
- ▶ Guards
- ▶ Déploiement sur GitHub Pages

PRÉSENTATION D'ANGULAR

- ▶ Angular est un framework (cadre de travail) Javascript. On en est à la version 19
- ▶ Permet de développer des sites web, des applications web, des applications mobiles hybrides, de manière robuste et efficace.
- ▶ Développé par Google en 2009
- ▶ AngularJS vs Angular 2+
- ▶ Principaux concurrents : **Vue.Js** - **React** - **Svelte** - **Ember**



PRÉSENTATION D'ANGULAR



SITE WEB

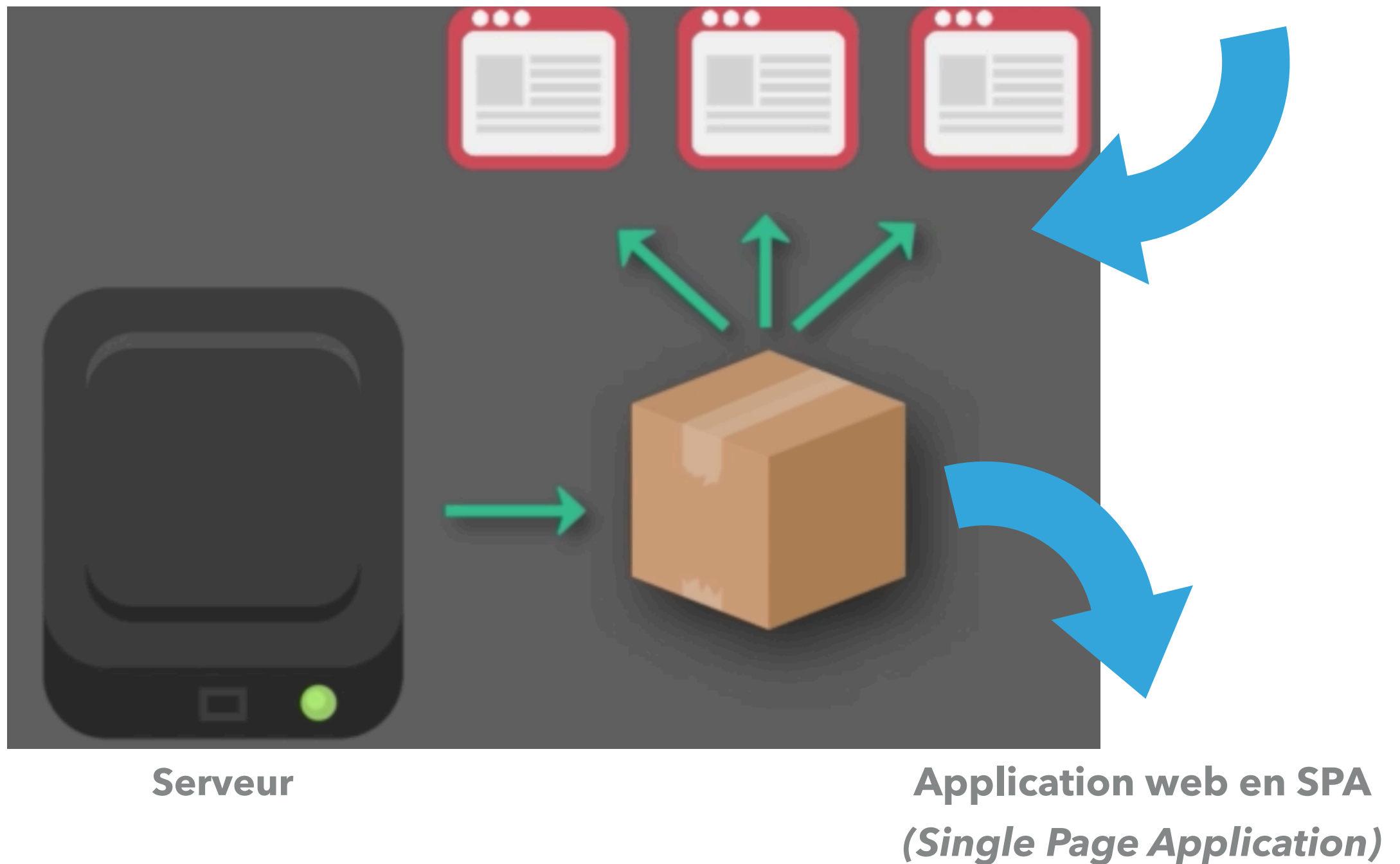


Serveur



APPLICATION WEB SPA

Intervention du JS

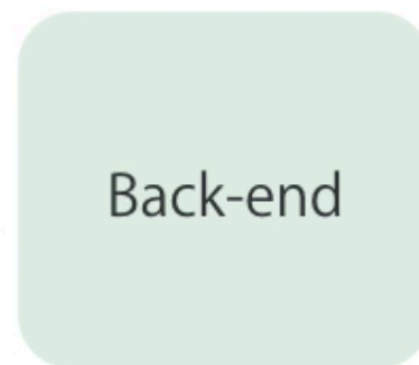


FRONT VS BACK

User Interface (UI)



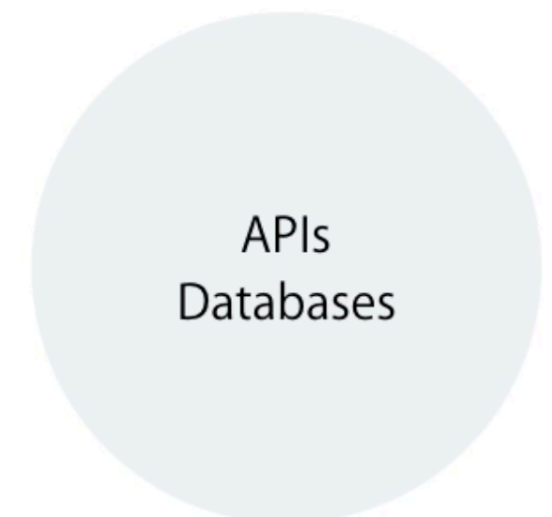
Data and Processing



Front-end Development



Back-end Development



PHILOSOPHIE

- ▶ Angular est un **orienté composants**. Une application Angular est donc un ensemble de petits composants assemblés entre-eux.
- ▶ Un composant est l'assemblage d'un morceau de code HTML et d'une classe JavaScript dédiée à une tâche particulière.
- ▶ Un composant est une partie complètement isolée de ton application, qui fonctionne de manière autonome.
- ▶ Ces composants reposent sur le standard des **web components** (pas encore supporté par tous les navigateurs). Ce standard a été pensé pour découper une page web en fonction de leurs rôles (navbar, discussion, etc...).

ILS UTILISENT ANGULAR



ECMAScript 6 .. ECMAScript 2015 .. ES6 .. JAVASCRIPT 6

- ▶ ECMAScript 6 est le nom de la dernière version standardisé de JavaScript.
- ▶ Cette standardisation a été approuvée par l'organisme de normalisation en Juin 2015.
- ▶ Le besoin d'une nouvelle standardisation s'est fait sentir pour fournir à JavaScript les moyens de développer des applications web robustes.
- ▶ ES6, via sa nouvelle syntaxe, apporte plusieurs nouvelles évolutions à JavaScript.



{ES6}

NOUVEAUTÉS DE ES6

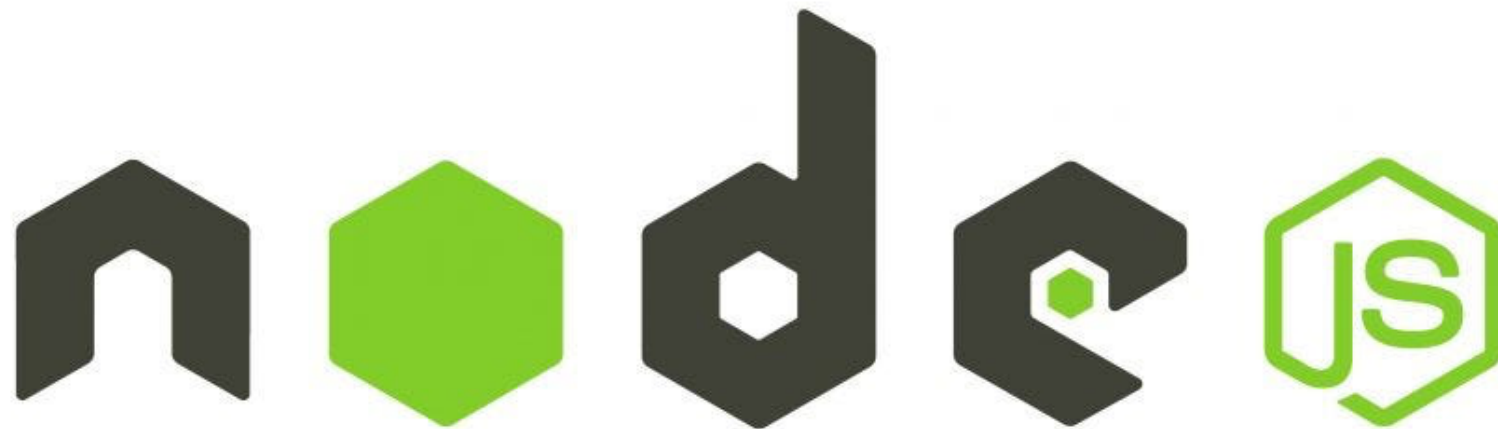
- ▶ Les **classes**
- ▶ Le nouveau mot-clef **let** : Le mot-clé let permet de déclarer une variable locale, dans le contexte où elle a été assignée.
- ▶ Le nouveau mot-clef **const**
- ▶ Les **fonctions fléchées** : Les fonctions fléchées, ou arrow functions, ne sont pas des fonctions classiques, parce qu'elles ne définissent pas un nouveau contexte comme les fonctions traditionnelles. Les **paramètres** de fonctions **par défaut**
- ▶ Le **template string** : on peut désormais utiliser des templates strings, qui commencent et se terminent par un **backtick** `.

TYPESCRIPT

- ▶ « *TypeScript est un langage de programmation libre et open-source, développé par Microsoft, qui a pour but d'améliorer et de sécuriser la production de code JavaScript. C'est un sur-ensemble de JavaScript (c'est-à-dire que tout code Javascript correct peut être utilisé avec TypeScript). Le code TypeScript est transcompilé en JavaScript, pouvant ainsi être interprété par n'importe quel navigateur web ou moteur JavaScript* ».
- ▶ Il est fortement recommandé d'utiliser TypeScript pour développer en Angular. C'est ce que Google recommande explicitement dans la documentation officielle d'Angular.
- ▶ En effet, Angular lui-même est développé avec TypeScript. C'est pour cela que nous utiliserons TypeScript dans ce cours.

ENVIRONNEMENT DE DÉVELOPPEMENT (1)

- ▶ Commençons par installer la dernière version de Node depuis **<https://nodejs.org/en/>**
- ▶ Node : Environnement d'exécution pour programmes en JS à l'extérieur du navigateur.
- ▶ Vérifier l'installation en tapant : `node --version`



ENVIRONNEMENT DE DÉVELOPPEMENT (2)

- ▶ L'installation du Node.js permettra d'utiliser Node Package Manager (**NPM**) pour pouvoir utiliser des librairies tierces telle que **Angular CLI (Command Line Interface)**.
- ▶ Angular CLI est un outil en ligne de commande, créé par l'équipe Google, pour démarrer rapidement un projet, déjà configuré, avec Webpack comme un outil de construction, des tests, du packaging, etc.
- ▶ Pour l'installer, il faut exécuter cette commande :

```
npm i -g @angular/cli
```

- ▶ Pour tester vérifier la version installée, tapez :

```
ng version
```


ENVIRONNEMENT DE DÉVELOPPEMENT

- ▶ Pour créer un nouveau projet Angular : `ng new nom_projet`
- ▶ Rajouter `--no-standalone` si vous êtes avec une version `>= 17`
- ▶ Lancez votre projet avec : `ng serve`
- ▶ Il nous reste à choisir un éditeur. Faites votre choix...

Visual Studio



Visual Studio 2017



Visual Studio Code



Visual Studio 2015

And More...



Sublime Text



Atom



Eclipse



Emacs



WebStorm



Vim

QUELQUES COMMANDES DU CLI

Commande	Utilisation
Component	<code>ng generate component my-new-component</code>
Directive	<code>ng g directive my-new-directive</code>
Pipe	<code>ng g pipe my-new-pipe</code>
Service	<code>ng g service my-new-service</code>
Class	<code>ng g class my-new-class</code>
Interface	<code>ng g interface my-new-interface</code>
Module	<code>ng g module my-module</code>

ARBORESCENCE D'UN PROJET

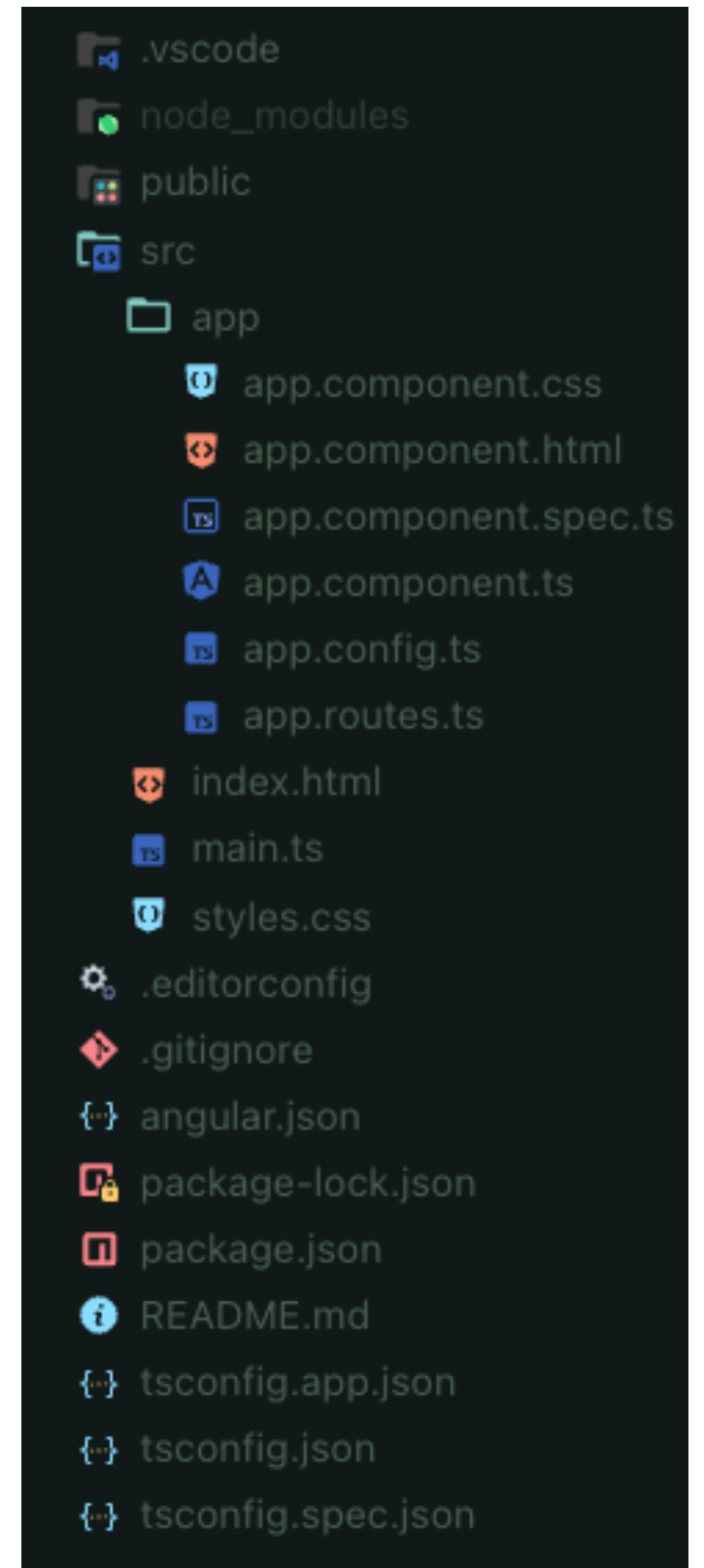
- ▶ **node_modules** : les dépendences déclarés dans package.json
- ▶ **src** : dossier contenant l'index, vos modules, le code source, les assets, les styles.
- ▶ **app** : Les sources du projet par défaut ainsi que le module `appModule`, le fichier `app.component.ts` ainsi que son template (`app.component.html`), son style (`app.component.css`) et `app.component.spec.ts` pour les tests unitaires.



```
> node_modules
✓ src
  ✓ app
    app.component.css
    app.component.html
    app.component.spec.ts
    app.component.ts
    app.module.ts
  > assets
  > environments
  ★ favicon.ico
  index.html
  main.ts
  polyfills.ts
  styles.css
  test.ts
  .browserslistrc
  .editorconfig
  .gitignore
  angular.json
  karma.conf.js
  package-lock.json
  package.json
  README.md
  tsconfig.app.json
  tsconfig.json
  tsconfig.spec.json
  tslint.json
```

ARBORESENCE D'UN PROJET STANDALONE

- ▶ **node_modules** : les dépendences déclarés dans package.json
- ▶ **public** : où stocker les fichiers statiques
- ▶ **src** : dossier contenant l'index, le code source, les styles globaux.
- ▶ **app** : renfermant le composant racine, le fichier du routing et le fichier de config.



ARBORESCENCE D'UN PROJET NGMODULES

- ▶ On a besoin au minimum d'un module racine et d'un composant racine par application.
- ▶ Le module racine se nomme par convention `AppModule`.
- ▶ Le composant racine se nomme par convention `AppComponent`.
- ▶ L'ordre de chargement de l'application est le suivant : `index.html` > `main.ts` > `app.module.ts` > `app.component.ts`
- ▶ Le fichier `package.json` initial est fourni avec des commandes prêtes à l'emploi comme la commande `npm start` (ou `ng serve`), qui nous permet de démarrer notre application web.

ANGULAR ET LES MODULES



- ▶ A la base, Angular est **modulaire**
 - ▶ Chaque application va définir Angular Modules ou NgModules
 - ▶ Chaque module Angular est une classe avec une annotation `@NgModule`
 - ▶ Chaque application a au moins un module, c'est le module principale.
- ▶ Rapide
- ▶ Orienté Composants

ANGULAR ET LES STANDALONES

- ▶ A partir de la version 15, Angular a introduit une nouvelle fonctionnalité appelée standalone components.
- ▶ Ce mode est désormais par défaut quand on crée une nouvelle application depuis Angular v17.
- ▶ Jusque-là, les composants devaient être déclarés dans des modules Angular, qui constituent un concept assez complexe à comprendre et utiliser correctement, surtout quand on démarre avec Angular.
- ▶ Utiliser des composants standalone nous permet d'éviter d'avoir à créer des modules Angular et rend les choses plus simples, particulièrement pour les débutants.

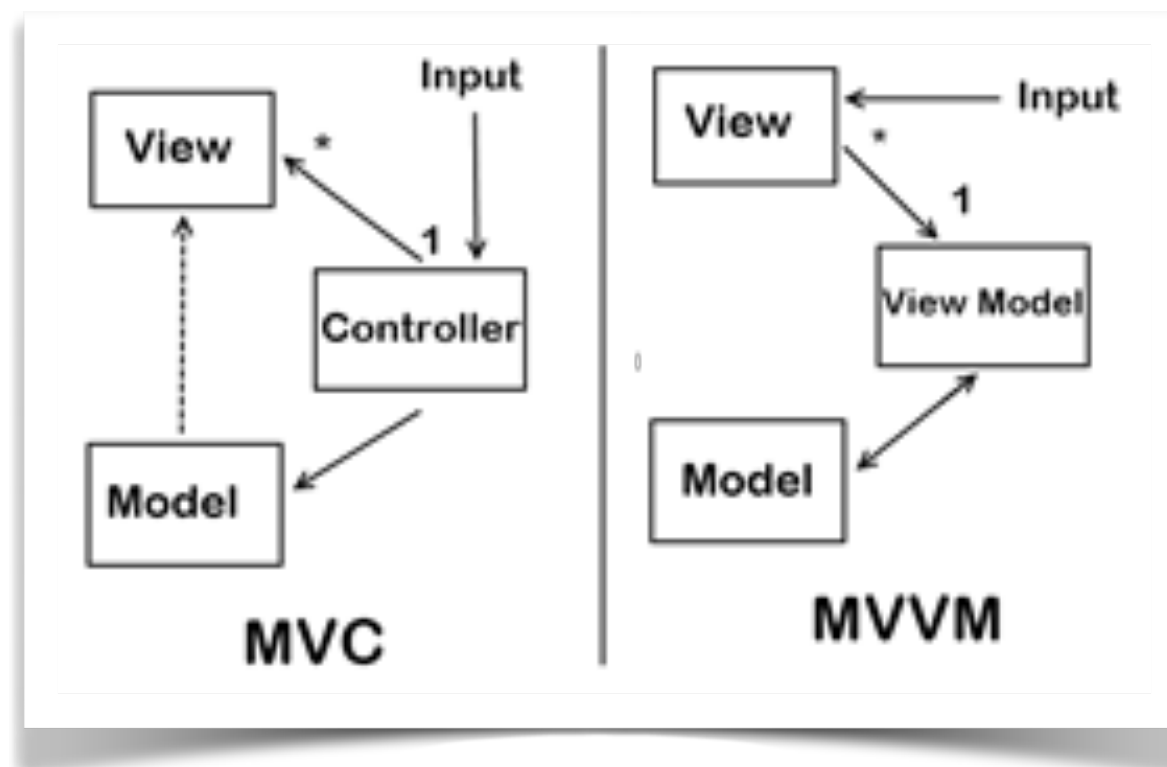


ARCHITECTURE

- ▶ Angular adopte principalement une architecture MVVM (Model-View-ViewModel), bien qu'il puisse parfois être confondu avec le MVC (Model-View-Controller) en raison de la flexibilité qu'il offre dans la structuration des applications.
- ▶ Dans l'architecture MVC :
 - ▶ **Model** : Contient les données et la logique métier.
 - ▶ **View** : Affiche les données.
 - ▶ **Controller** : Gère les interactions utilisateur et met à jour le Modèle ou la Vue en conséquence.
- ▶ Angular ne respecte pas strictement cette séparation.

ARCHITECTURE

- ▶ En effet :
- ▶ Le composant Angular remplit souvent un double rôle de "Controller" et de "ViewModel".
- ▶ La liaison des données (via @Input, @Output, ou [(ngModel)]) intègre des mécanismes de gestion des états qui s'éloignent du paradigme MVC classique.

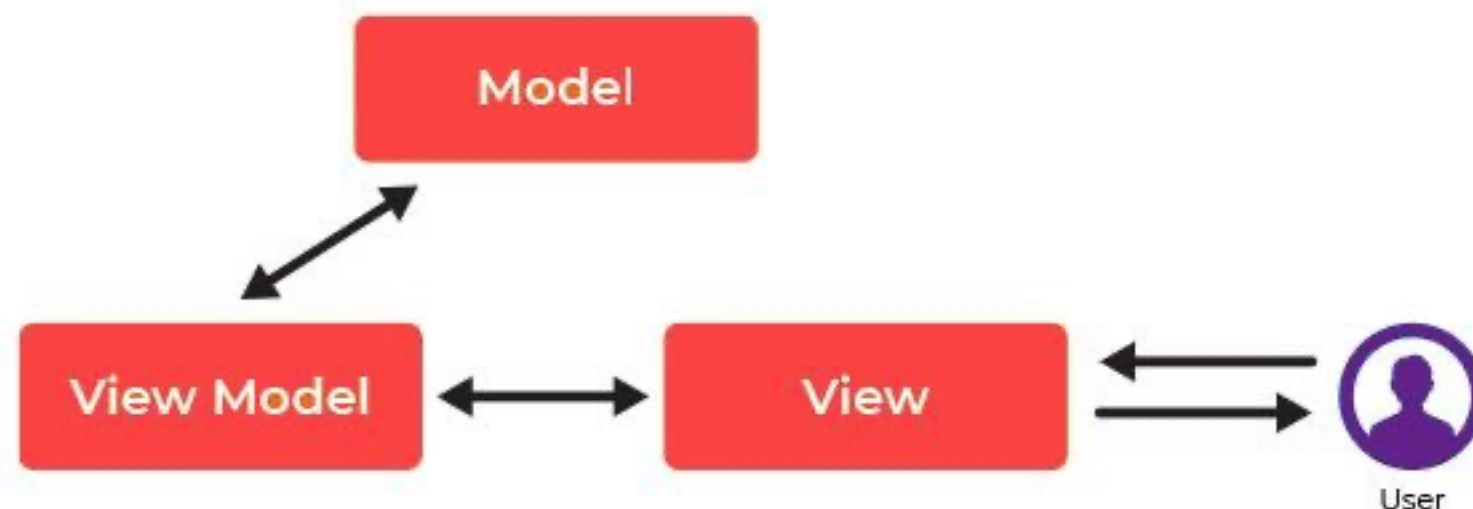


ARCHITECTURE

- ▶ Dans l'architecture MVVM :
 - ▶ **Model** : Représente les données brutes ou la logique métier, souvent gérée dans Angular à travers les **services** et les **modèles de données**.
 - ▶ **View** : Correspond à la partie visible de l'application, définie par les **templates HTML** et les directives Angular.
 - ▶ **ViewModel** : Agit comme un médiateur entre la Vue et le Modèle, encapsulant la logique de présentation et permettant la liaison bidirectionnelle. En Angular, cela correspond généralement au **composant**.
- ▶ Ainsi, il est clair qu'en Angular, le composant agit comme un pont (intermédiaire), facilitant la liaison bidirectionnelle.

ARCHITECTURE

- ▶ De plus, Angular adopte des concepts qui renforcent son orientation MVVM :
- ▶ **Data Binding** : La liaison bidirectionnelle (`[[ngModel]]`) est une caractéristique centrale de MVVM.
- ▶ **Directives** : Elles permettent de manipuler la Vue directement sans affecter la logique du Modèle.
- ▶ **Services** : Ils encapsulent la logique métier et restent découplés du ViewModel, ce qui est cohérent avec MVVM.



AJOUTER BOOTSTRAP À VOTRE PROJET

- ▶ On peut ajouter Bootstrap de plusieurs façons. La plus recommandée est la suivante :

1- Via npm avec la commande : `npm install bootstrap --save`

2- Ensuite, on ajoute dans le chemin des dépendances dans les tableaux styles et scripts dans le fichier angular.json:

```
"styles": [  
  "../node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "styles.css",  
]
```

AJOUTER BOOTSTRAP À VOTRE PROJET

- ▶ *2ème méthode* : Vous pouvez aussi ajouter dans le fichier `src/style.css` un import de vos bibliothèques.
- `@import "~bootstrap/dist/css/bootstrap.min.css";`
- ▶ *3ème méthode* : Ajouter l'appel au bootstrap donc le head de `index.html`
- ▶ Essayer la même chose avec *font-awesome* qu'on pourra également utiliser dans notre projet.